1 Introduction and Overview

1.1 DEFINITIONS

Before going to study the actual subject, it will be a special advantage if we aware ourselves about the various terms involved in the subject. In the present section, basic terminologies and concepts are described with examples.

Data

Data means value or set of values. In both the singular and plural form, this term is data. Following are some examples of data:

(i) 34
(ii) 12/01/1965
(iii) ISBN 81-203-0000-0
(iv) | | |
(v) Pascal
(vi) Æ
(vii) 21, 25, 28, 30, 35, 37, 38, 41, 43

In all the above examples, some values are represented in different ways. Each value or collection of all such values is termed as data.

Entity

An *entity* is one that has certain attributes and which may be assigned values. For example, an employee in an organisation is an entity. The possible attributes and the corresponding values for an entity in the present example are:

Entity :	EMPLOYEE			
Attributes :	NAME	DOB	SEX	DESIGNATION
Values :	RAVI ANAND	30/12/61	M	DIRECTOR

All the employees in an organisation constitute an *entity set*. Each attribute of an entity set has a range of values, and is called the *domain* of attribute. Domain is the set of all possible values that could be assigned to the particular attribute. For example, in the EMPLOYEE entity, the attribute SEX has domain as {M, F}. It can be noted that, an entity after assigning its values results a composite data.

Information

The term *information* is used for data with its attribute(s). In other words, information can be

Copyrighted material

defined as meaningful data or processed data. For example, a set of data is presented during defining the term data, all these data become information if we impose the meaning as mentioned below related to a person:

Data	*Meaning*
34	Age of the person
12/01/1965	Date of birth of the person
ISBN 81-203-0000-0	Book number, recently published by the person
I I I	Number of awards achieved by the person in tally mark
Pascal	Nick name of the person
Æ	Signature of the person
21, 25, 28, 30, 35, 37, 38, 41, 43	Important ages of the person

Difference between data and information

From the definition of data and information it is evident that these two are not the same, however, there is a relation between them. Figure 1.1 shows the interrelation between data and information:

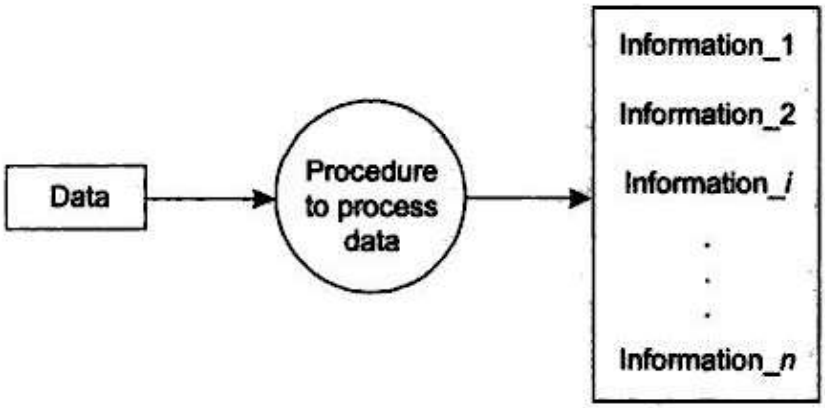


Fig. 1.1 Relation between data and information.

As an example, suppose there is a set of data comprising of the amount of milk consumed by a person in a month. From this given set of data information can be retrieved are as follows:

(i) What is the total amount of milk consumed?
(ii) In which day maximum milk is consumed?
(iii) In which day minimum milk is consumed?
(iv) What is the average amount of milk consumption per day?
(v) What amount of carbohydrate is assimilated?
(vi) What amount of protein is assimilated?
(vii) What amount of fat is assimilated?
(viii) What amount of mineral is assimilated?
(ix) What amount of average calorie is gained per day?

and so on. To get these information from the given set of data, we are to define the processes

Copyrighted material

and then to apply the corresponding process on the data. (In fact, the terms, *data*, *entity* and *information* have no standard definition in computer science and they are often used interchangeably.)

Data type

A *data type* is a term which refers to the kind of data that may appear in computation. Following are a few well-known data types (*see* table):

Data	*Data type*
34	Numeric (integer)
12/01/1965	Date
ISBN 81-203-0000-0	Alphanumeric
| | |	Graphics
Pascal	String
Æ	Image
21, 25, 28, 30, 35, 37, 38, 41, 43	Array of integers

Real, boolean, character, complex, etc., are also some more frequently used data types.

Built-in data type

With every programming language, there is a set of data types called *built-in data types*. For example, in C, FORTRAN and Pascal, the data types that are available as built-in are listed below:

C : int, float, char, double, enum etc.

FORTRAN : INTEGER, REAL, LOGICAL, COMPLEX, DOUBLE PRECISION, CHARACTER, etc.

Pascal : Integer, Real, Character, Boolean, etc.

With the built-in data types, programming languages allow users a lot of advantages regarding the processing of various types of data. For example, if a user declares a variable of type Real (say), then several things are automatically implied, such as how to store a value for that variable, what are the different operations possible on that type of data, what amount of memory is required to store, etc. All these things are taken care by the compiler or run-time system manager.

Abstract data type

When an application requires a special kind of data which is not available as built-in data type then it is the programmer's burden to implement his own kind of data. Here, the programmer has to give more effort regarding how to store value for that data, what are the operations that meaningfully manipulate variables of that kind of data, amount of memory require to store for a variable. The programmer has to decide all these things and accordingly implement it. Programmers' own data type is termed as *abstract data type*. Abstract data type is also alternatively termed as *user-defined data type*. For example, we want to process dates of the form dd/mm/yy. For

Copyrighted material

this, no built-in data type is known in C, FORTRAN, and Pascal. If a programmer wants to process dates, then an abstract data type, say Date, can be devised and various operations like: to add few days to a date to obtain another date, to find the days between two dates, to obtain a day for a given date, etc., have to be defined accordingly. Besides these, programmers should decide how to store the data, what amount of memory will be needed to represent a value of Date, etc. An abstract data type, in fact, can be built with the help of built-in data types and other abstract data type(s) already built by the programmer. Some programming languages allow facility to build abstract data type easily. For example, using struct/class in C/C++, and using record in Pascal, programmers can define their own data types.

1.2 CONCEPT OF DATA STRUCTURES

Digital computer can manipulate only primitive data, that is, data in terms of 0's and 1's. Manipulation of primitive data is inherent within the computer and need not require any extra effort from the user side. But in our real life applications, various kind of data other than the primitive data are involved. Manipulation of real-life data (can also be termed as user data) requires the following essential tasks:

1. Storage representation of user data: user data should be stored in such a way that computer can understand it.
2. Retrieval of stored data: data stored in a computer should be retrieved in such a way that user can understand it.
3. Transformation of user data: various operations which require to be performed on user data so that it can be transformed from one form to another.

Basic theory of computer science deals with the manipulation of various kinds of data, wherefrom the concept of data structures comes. In fact, data structure is the fundamentals in computer science. For a given kind of user data, its structure implies the following:

1. Domain ($\mathcal{D}$): This is the range of values that the data may have. This domain is also termed as data object.
2. Function ($\mathcal{F}$): This is the set of operations which may legally be applied to elements of data object. This implies that for a data structure we must specify the set of operations.
3. Axioms ($\mathcal{A}$): This is the set of rules with which the different operation belongs to $\mathcal{F}$ actually can be implemented.

Now we can define the term data structure.

A *data structure* D is a triplet, that is, D = ($\mathcal{D}$, $\mathcal{F}$, $\mathcal{A}$) where $\mathcal{D}$ is a set of data object, $\mathcal{F}$ is a set of functions and $\mathcal{A}$ is a set of rules to implement the functions. Let us consider an example.

We know for the integer data type (int) in C programming language the structure includes of the following type:

$\mathcal{D}$ = (0, ±1, ±2, ±3, ...)

$\mathcal{F}$ = (+, −, *, /, %)

$\mathcal{A}$ = (A set of binary arithmetics to perform addition, subtraction, division, multiplication, and modulo operations.)

Copyrighted material

It can be easily revealed that the triplet ($\mathcal{D}$, $\mathcal{F}$, $\mathcal{A}$) is nothing but an abstract data type. Also, the elements in set $\mathcal{D}$ are not necessarily from primitive data it may contain from some other abstract data type. Alternatively, an implementation of a data structure D is a mapping from $\mathcal{D}$ to a set of other data structures D_i, $i = 1, 2, ..., n$, for some n. More precisely, this mapping specifies how every object of $\mathcal{D}$ is to be represented by the objects of D_i, $i = 1, 2, ..., n$. Every function of D must be written using the function of the implementing data structures D_i, $i = 1, 2, ..., n$. The fact is that each of the implementing data structures is either primitive data type or abstract data type. We can conclude the discussion with another example.

Suppose, we want to implement a data type namely `complex` as abstract data type. Any variable of complex data type has two parts: real part and imaginary part. In our usual notation, if z is a complex number then $z = x + iy$, where x and y are real and imaginary parts respectively. Both x and y are of `Real` data type which is another abstract data type (available as built-in data type). So, abstract data type `complex` can be defined using the data structure `Real` as:

```
Complex z {
  x : Real
  y : Real
}
```

Now the set $\mathcal{D}$ of `Complex` can be realised from the domain of x and y which is `Real` in this case. Let us specify the set of operations for the `Complex` data type, which are stated as $\mathcal{F}$:

$$\mathcal{F} = (\oplus, \ominus, \otimes, \oplus, \nabla, \|)$$

Assume that $z_1 = x_1 + iy_1$ and $z_2 = x_2 + iy_2$ are two data of `Complex` data type. Then we can define the rules for implementing the operations in $\mathcal{F}$ giving thus axioms $\mathcal{A}$. In the current example, for `Complex` data type:

$\mathcal{A} = \{$

$z = z_1 \oplus z_2 = (x_1 + x_2) + i(y_1 + y_2)$ (complex addition)

$z = z_1 \ominus z_2 = (x_1 - x_2) + i(y_1 - y_2)$ (complex subtraction)

$z = z_1 \otimes z_2 = (x_1 \times x_2 - y_1 \times y_2) + i(x_1 \times y_2 + x_2 \times y_1)$ (complex multiplication)

$$z = z_1 \oplus z_2 = \frac{x_1 \times x_2 + y_1 \times y_2}{x_2^2 + y_2^2} + i\,\frac{x_2 \times y_1 - x_1 \times y_2}{x_2^2 + y_2^2}$$ (complex division)

$$z = \nabla z_1 = \frac{x_1}{x_1^2 + y_1^2} - i\,\frac{y_1}{x_1^2 + y_1^2}$$ (complex conjugate)

$$Z = |z_1| = \sqrt{x_1^2 + y_1^2}$$ (complex magnitude)

$\}$

Note that, how different operations of `Complex` data type can be implemented using the operations $+, -, \times, /, \sqrt{\ }$, of implementing data structure, namely `Real`.

Copyrighted material

Assignment 1.1 Implement *Date* as an abstract data type which comprises of dd/mm/yy, where dd varies from 1 to 31, mm varies from 1 to 12 and yy is any integer with four digits.

Specify $\mathcal{F}$ consisting of all possible operations on variables of type *Date* and then define $\mathcal{A}$ to implement all the operations in $\mathcal{F}$. Assume the implementing data structure(s) which is/are necessary.

1.3 OVERVIEW OF DATA STRUCTURES

In computer science, several data structures are known depending on area of applications. Of them, few data structures are there which are frequently used almost in all application areas and with the help of which almost all complex data structure can be constructed. These data structures are known as *fundamental* data structures or *classic* data structures. Figure 1.2 gives a classification of all classic data structures.

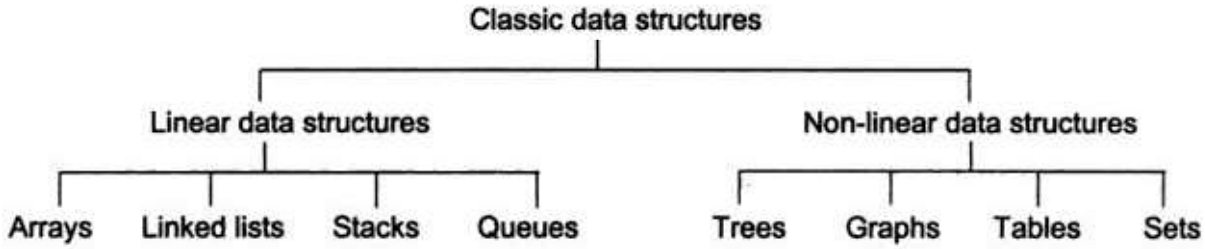


Fig. 1.2 Classification of classic data structures.

In addition to these classic data structures, other data structures such as lattice, Petri nets, neural nets, search graphs, semantic nets, etc., are known in various applications. These are known to be very complex data structures. (Discussion of all these complicated data structures is beyond the scope of this book.)

It can be observed that all the classic data structures are classified into two main classes: *linear data structures* and *non-linear data structures*. In case of linear data structures, all the elements form a sequence or maintain a linear ordering. On the other hand, no such sequence in elements, rather all the elements are distributed over a plane in case of non-linear data structures. Again within each classical data structure there are a number of variations as depicted in Figure 1.2. In this book, we have planned for eight chapters to discuss eight classic data structures separately. As an overview, all the classic data structures are primarily introduced which can be seen in Figure 1.3.

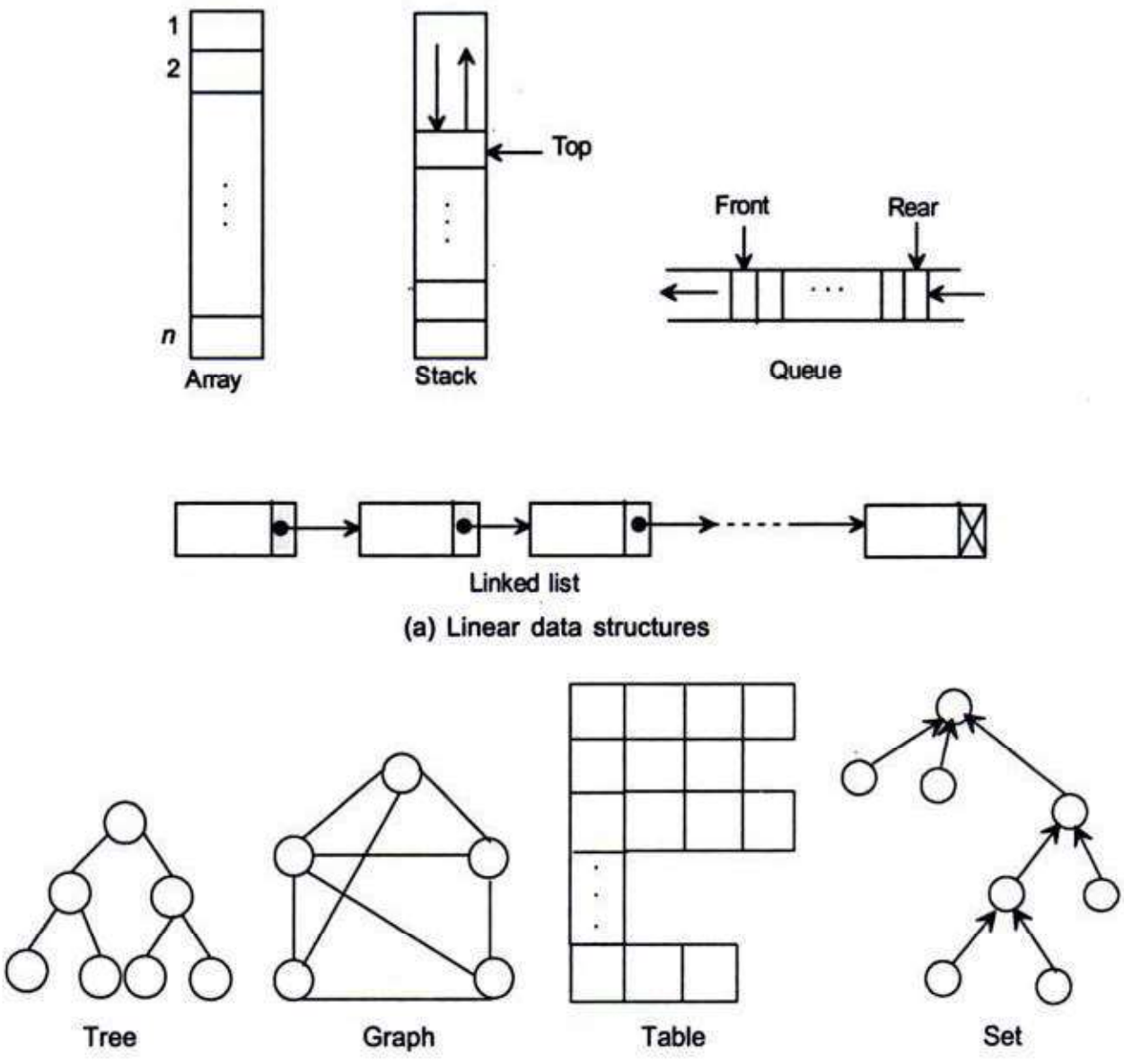
Assignment 1.2 Devise a data structure (say Graphic) to represent a graphics data as shown in Figure 1.4. Also, define the following operation on any data of this type:

(a) ScaleUp(P): To magnify the image by certain per cent P

(b) ScaleDn(P): To reduce the image by certain per cent P

(c) Rotate(R, A): To rotate the image either clockwise (R = +) or anticlockwise (R = −) by a certain angle A.

Copyrighted material



(a) Linear data structures

(b) Non-linear data structures

Fig. 1.3 Overview of classic data structures.

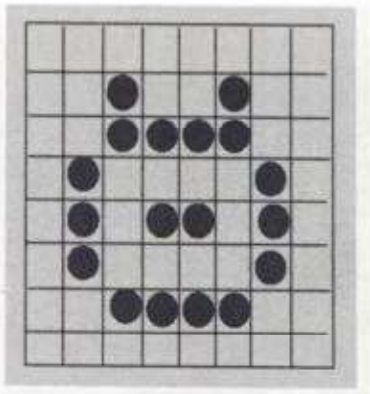


Fig. 1.4

Copyrighted material

1.4 IMPLEMENTATION OF DATA STRUCTURES

We will implement all the classic data structures as mentioned in the previous section in two phases:

Phase 1

Storage representation: Here we have decided how a data structure can be stored in computer memory. This storage representation, in general, is based on the use of other data structures.

Phase 2

Algorithmic notation of various operations of the classic data structure: Function for manipulating a data structure is expressed in terms of algorithms so that details of the operation can be understood easily and reader can implement them with the help of any programming language. It will be preferable to use C/C++ programming because of their versatile features and modern programming approaches.

In order to express the algorithm for a given operation, we will assume different control structures and notations as stated below:

Algorithm <Name of the operation>(Input parameters; Output parameters)
Input: <Specification of input data for the operation>
Output: <Specification of output after the successful performance of the operation>
Data structure: <If the operation assumes other data structure for its implementation>

Steps:

1. . . . . .
2. **If** <condition> **then**
   1. . . . . .
   2. . . . . .
   . . .
   . . .
3. **Else**
   1. . . . . .
   2. . . . . .
   . . .
   . . .
4. **EndIf**
5. . . .
6. **While** <condition> **do**
   1. . . . . .
   2. . . . . .
   . . .
   . . .
7. **EndWhile**
8. . . .

Copyrighted material

9. **For** <loop condition> **do**
   1. . . . . .
   2. . . . . .
   . . .
   . . .
10. **EndFor**
11. . . .
12. **Stop**

Different statements that will be used in the algorithms are very much similar to the statement in C language. Sufficient comment will be introduced side by side against statements to indicate what step that a statement will perform. The convention of putting comment is similar to the convention of placing comments in C/C++ programming language.

Another convention that we will assume is the naming of variables. The variable names are either in capital or small letters. The variable names in capital letters will be treated as either constant or as input to the algorithm. All the variable names in small letters are local to the algorithm and will be treated as temporary variables.

'Problems to Ponder' at the end of each chapter includes problems related to the topics discussed in the concerned chapter. In addition to, a number of 'Assignments' are also provided during the discussion in each chapter. Students can improve their level of learning if they practise these assignments and solve problems.

Copyrighted material

2 Arrays

If we want to store a group of data together in one place then array is the one data structure we are looking for. This data structure enables us to arrange more than one element, that is why it is termed as composite data structure. In this data structure, all the elements are stored in contiguous locations of memory. Figure 2.1 shows an array of data stored in a memory block starting at location 453.

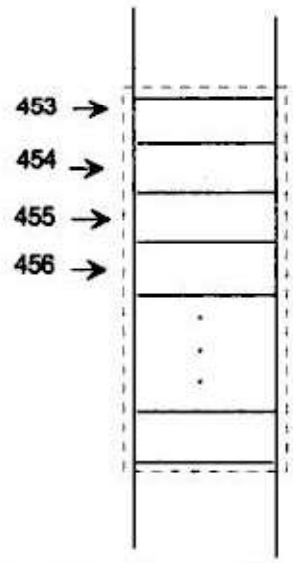


Fig. 2.1 Array of data.

2.1 DEFINITION

An *array* is a finite, ordered and collection of homogeneous data elements. Array is finite because it contains only limited number of elements; and ordered, as all the elements are stored one by one in contiguous locations of computer memory in a linear ordered fashion. All the elements of an array are of the same data type (say, integer) only and hence it is termed as collection of homogeneous elements. Following are some examples:

1. An array of integers to store the age of all students in a class.
2. An array of strings (of characters) to store the name of all villagers in a village.

Note: 1. An array is known as linear data structure because, all elements of the array are stored in a linear order.

2. The data structure is generally a built-in data type in all programming languages. Declaration syntax for an array A, say, of 100 integers in BASIC, FORTRAN, Pascal and C are as below:

Copyrighted material

```
BASIC    : DIMENSION A[100]
FORTRAN  : DIM A[100]
Pascal   : A: ARRAY[1...100] of integer
C        : int A[100]
```

2.2 TERMINOLOGY

Size. Number of elements in an array is called the *size* of the array. It is also alternatively termed as *length* or *dimension*.

Type. *Type* of an array represents the kind of data type it is meant for. For example, array of integers, array of character strings, etc.

Base. *Base* of an array is the address of memory location where the first element in the array is located. For example, 453 is the base address of the array as mentioned in Figure 2.1.

Index. All the elements in an array can be referenced by a subscript like A_i or A[i], this subscript is known as *index*. Index is always an integer value. As each array elements is identified by a subscript or index that is why an array element is also termed as *subscripted* or *indexed variable*.

Range of index. Indices of array elements may change from a lower bound (L) to an upper bound (U), which are called the boundaries of an array.

In a declaration of an array in FORTRAN (DIMENSION A[100]), range of index is 1 to 100. For the same array in C (int A[100]) the range of index is from 0 to 99. These are all default range of indices. However in Pascal, a user can define the range of index for any lower bound to upper bound, for example, for A: ARRAY[–5...19] of integer, the points of the range is –5, –4, –3, ..., 18, 19. Here, the index of i-th element is $-5 + i - 1$. In terms of L, the lower bound, this formula stands as:

$$\text{Index } (A_i) = L + i - 1$$

If the range of index varies from L ... U then the size of the array can be calculated as

$$\text{Size } (A) = U - L + 1$$

Word. Word denotes the size of an element. In each memory location, computer can store an element of word size w, say. This word size varies from machine to machine such as 1 byte (in IBM CPU-8085) to 8 bytes (in Intel Pentium PCs). Thus, if the size of an element is double the word size of a machine then to store such an element, it requires two consecutive memory locations.

2.3 ONE-DIMENSIONAL ARRAY

If only one subscript/index is required to reference all the elements in an array then the array will be termed as *one-dimensional* array or simply an array.

Copyrighted material

2.3.1 Memory Allocation for an Array

Memory representation of an array is very simple. Suppose, an array A[100] is to be stored in a memory as in Figure 2.2. Let the memory location where the first element can be stored is M. If each element requires one word then the location for any element say A[i] in the array can be obtained as:

$$\text{Address (A}[i]) = M + (i - 1)$$

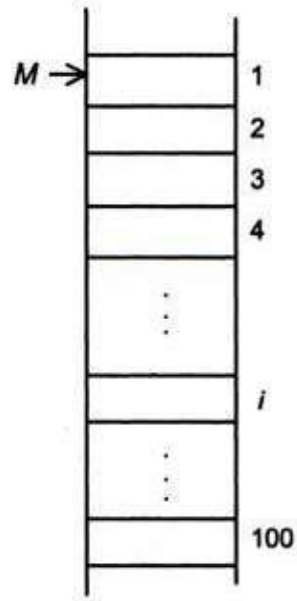


Fig. 2.2 Physical representation of a one-dimensional array.

Likewise, in general, an array can be written as A[L ... U], where L and U denote the lower and upper bounds for index. If it is stored starting from memory location M, and for each element it requires w number of words, then the address for A[i] will be

$$\text{Address (A}[i]) = M + (i - L) \times w$$

The above formula is known as *indexing formula*; which is used to map the logical presentation of an array to physical presentation. By knowing the starting address of an array M, the location of i-th element can be calculated instead of moving towards i from M, Figure 2.3.

Assignment 2.1 Suppose, an array A [–15 ... 64] is stored in a memory whose starting address is 459. Assume that word size for each element is 2. Then obtain the following:

(a) How many number of elements are there in the array A?

(b) If one word of the memory is equal to 2 bytes, then how much memory is required to store the entire array?

(c) What is the location for A[50]?

(d) What is the location of 10-th element?

(e) Which element is located at 589?

Copyrighted material

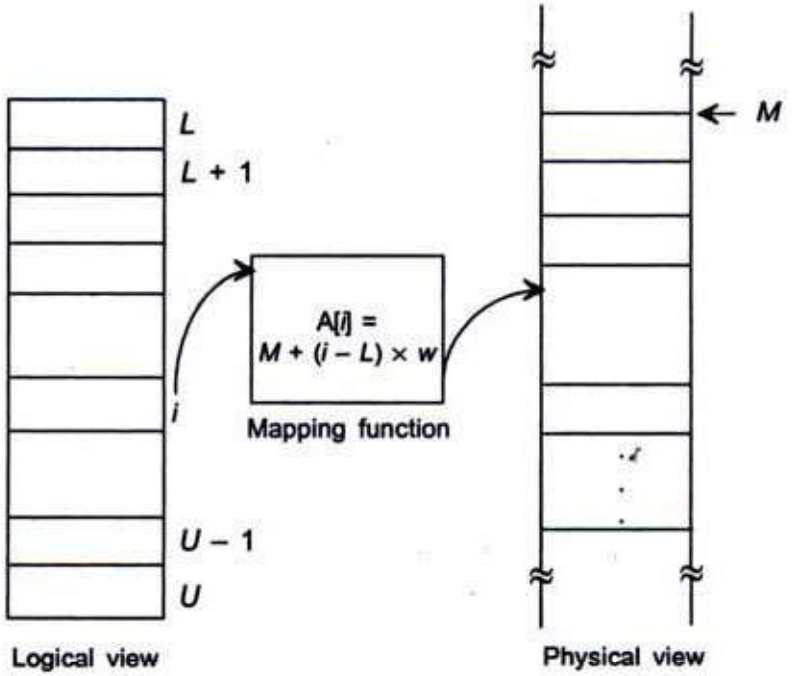


Fig. 2.3 Address mapping between logical and physical views of an array.

Assignment 2.2 In the indexing formula in Section 2.3.1, we have taken that the array is stored from lower region of the memory to the higher region. But in some computer the convention is just reverse, that is, starting an array i at higher region and hence Address (A_i) > Address (A_{i+1}) for all $L < i < U$. Modify the indexing formula for this case.

2.3.2 Operations on Arrays

Various operations that can be performed on an array are: traversing, sorting, searching, insertion, deletion, merging.

Traversing

This operation is used visiting all elements in an array. A simplified algorithm is presented as below:

Algorithm TRAVERSE_ARRAY( )
Input: An array A with elements.
Output: According to PROCESS( ).
Data structures: Array A[L ... U]. //L and U are the lower and upper bound //of array index

Steps:
1. $i = L$ //Start from first location L
2. While $i \leq U$ do
   1. PROCESS(A[i])
   2. $i = i + 1$ //Move to the next location
3. EndWhile
4. Stop

Copyrighted material

Note: Here PROCESS( ) is a procedure which when called for an element can perform an action. For example, display the element on the screen, determine whether A[*i*] is empty or not, etc. PROCESS( ) also can be used to manipulate some special operations like count the special element of interest (for example, negative numbers in an integer array), updating of each element (say, increase each by 10%) and so on.

Sorting

This operation, if performed on an array, will sort it in a specified order (ascending/descending). The following algorithm is used to store the elements of an integer array in ascending order.

Algorithm SORT_ARRAY( )
Input: An array with integer data.
Output: An array with sorted elements in an order according to ORDER( ).
Data structures: An integer array A[*L*...*U*]. //*L* and *U* are the lower and upper bound of //array index

Steps:

```
1. i = U
2. While i ≥ L do
   1. j = L                                  //Start comparing from first
   2. While j < i do
      1. If ORDER(A [j], A[j + 1]) = FALSE   //If A[j] and A[j + 1] are not in order
         1. SWAP(A[j], A[j + 1])             //Interchange the elements (see Figure 2.4)
      2. EndIf
      3. j = j + 1                           //Go to next element
   3. EndWhile
   4. i = i − 1
3. EndWhile
4. Stop
```

Here, ORDER(...) is a procedure to test whether two elements are in order and SWAP(...) is a procedure to interchange the elements between two consecutive locations.

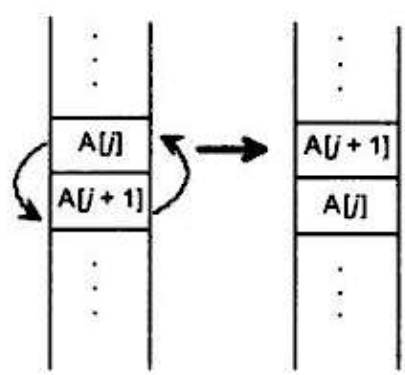


Fig. 2.4 Swapping of two elements in an array.

Copyrighted material

Assignment 2.3 Write algorithms to

(a) Sort an array of integers in ascending order.
(b) Sort an array of integers in descending order.
(c) Sort an array of string of characters in lexicographic order.
(d) Sort an array of an abstract data type.

Hint: You have to think of only little modification of procedure ORDER(...) in each case.

Searching

This operation is applied to search an element of interest in an array. The simplified version of the algorithm is as follows:

Algorithm SEARCH_ARRAY(KEY)
Input: KEY is the element to be searched.
Output: Index of KEY in A or a message on failure.
Data structures: An array A[L ... U]. //L and U are the lower and upper bound of array index

Steps:

1. $i = L$, found = 0, location = 0 //found = 0 indicates search is not finished and unsuccessful
2. While ($i \leq U$) and (found = 0) do // Continue if all or any one condition do(es) not satisfy
   1. If COMPARE(A[i], KEY) = TRUE then //If key is found
      1. found = 1 //Search is finished and successful
      2. location = i
   2. Else
      1. $i = i + 1$ //Move to the next
   3. EndIf
3. EndWhile
4. If found = 0 then
   1. Print "Search is unsuccessful : KEY is not in the array"
5. Else
   1. Print "Search is successful : KEY is in the array at location", location
6. EndIf
7. Return(location)
8. Stop

Assignment 2.4 In the algorithm discussed above assume that elements are randomly distributed. Modify the algorithm to give more faster algorithm when the elements are already in sorted (either ascending or descending) order.

Insertion

This operation is used to insert an element into an array provided that the array is not full (*see* Figure 2.5). Let us observe the algorithmic description for the insertion operation.

Copyrighted material

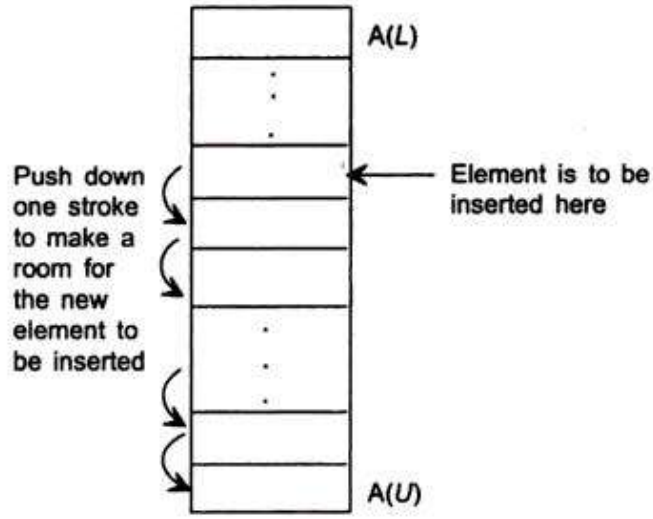


Fig. 2.5 Insertion of an element to an array.

Algorithm INSERT(KEY, LOCATION)
Input: KEY is the item, LOCATION is the index of the element where it is to be inserted.
Output: Array enriched with KEY.
Data structures: An array $A[L \ldots U]$. //L and U are the lower and upper bound of array index

Steps:

1. If A[U] ≠ NULL then
   1. Print "Array is full: No insertion possible"
   2. Exit //End of execution
2. Else
   1. $i = U$ //Start pushing from bottom
   2. While i > LOCATION do
      1. A[i + 1] = A[i]
      2. $i = i - 1$
   3. EndWhile
   4. A[LOCATION] = KEY //Put the element at desired location
   5. $U = U + 1$ //Update the upper index of the array
3. EndIf
4. Stop

Assignment 2.5 The algorithm INSERT only checks the last element for vacancy. But an array may be empty from any i-th position ($L \le i \le U$); in that case number of push down can be reduced instead of pushing down the entire trailing part. Modify the above algorithm INSERT when the last element is at i-th location. ($i \le U$).

Assignment 2.6 How can an empty array be defined? Verify that whether all the algorithms discussed so far in this chapter can work with empty array. If not, modify algorithm(s) so that they can work even for empty array(s).

Copyrighted material

Deletion

This operation is used to delete a particular element from an array. The element will be deleted by overwriting it with its subsequent element and this subsequent element then is to be deleted. In other words, push the tail (the part of the array after the elements which are to be deleted) one stroke up. Consider the following algorithm to delete an element from an array:

Algorithm DELETE(KEY)
Input: KEY the element to be deleted.
Output: Slimed array without KEY.
Data structures: An array A[*L*...*U*]. //*L* and *U* are the lower and upper bound of //array index

Steps:

1. i = SEARCH_ARRAY(A, KEY) //Perform the search operation on A and return //the location
2. If (i = 0) then
   1. Print "KEY is not found: No deletion"
   2. Exit //Exit the program
3. Else
   1. While $i < U$ do
      1. A[i] = A[i + 1] //Replace the element by its successor
      2. $i = i + 1$
   2. EndWhile
4. EndIf
5. A[U] = NULL //The bottom most element is made empty //(*see* Figure 2.6)
6. $U = U - 1$ //Updated the upper bound now
7. Stop

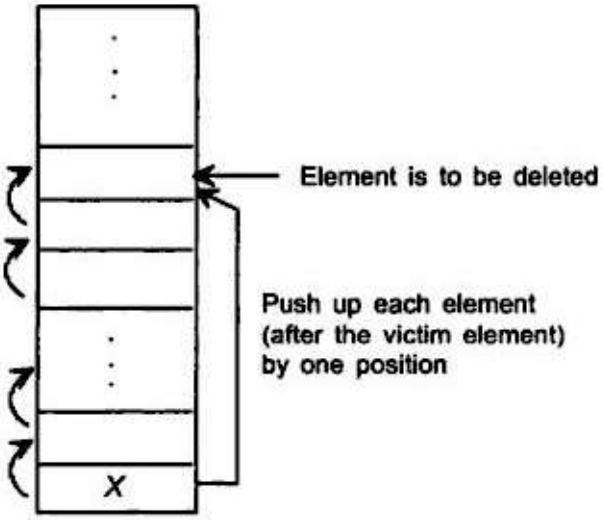


Fig. 2.6 Deletion of an element from an array.

Note: It is a general practice that no intermediate location will be made empty, that is, an array should be packed and empty locations are at the tail of an array.

Copyrighted material

Merging

Merging is an important operation when we need to compact the elements from two different arrays into a single array (*see* Figure 2.7).

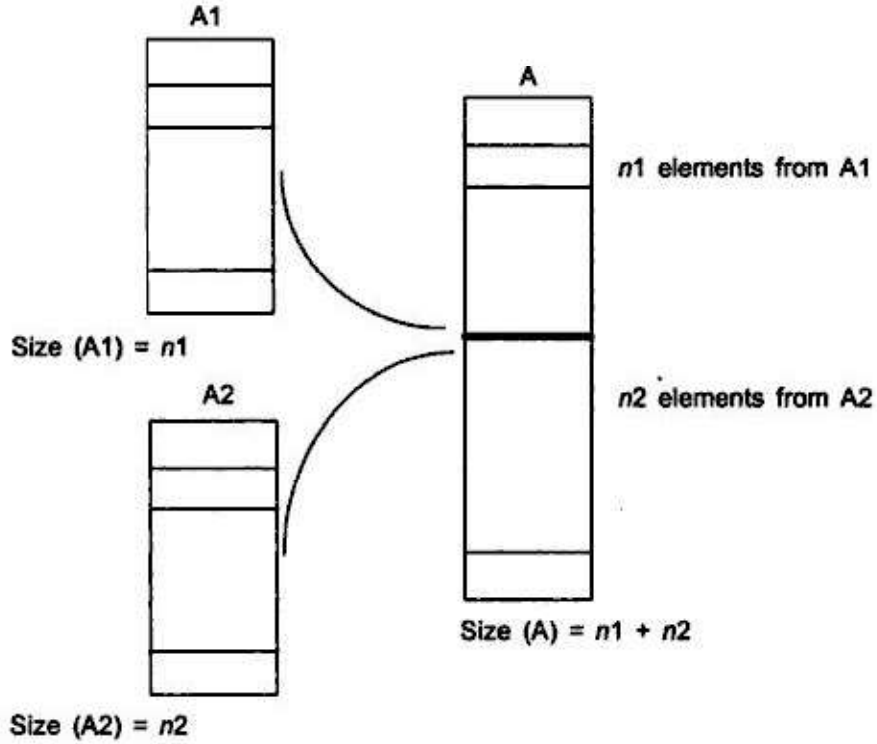


Fig. 2.7 Merging of A_1 and A_2 to A.

Algorithm MERGE(A_1, A_2; A)

Input: Two arrays $A_1[L_1 \ldots U_1]$, $A_2[L_2 \ldots U_2]$.

Output: Resultant array $A[L \ldots U]$, where $L = L_1$, and $U = U_1 + (U_2 - L_2 + 1)$ when A_1 is appended after A_2.

Data structures: Array structure.

Steps:

1. $i_1 = L_1$, $i_2 = L_2$, //Initialization of control variables
2. $L = L_1$, $U = U_1 + U_2 - L_2 + 1$ //Initialization of lower and upper bounds of //resultant array A
3. $i = L$
4. Allocate memory for $A[L \ldots U]$
5. While $i_1 \leq U_1$ do //To copy array A_1 into the first part of A
   1. $A[i] = A_1[i_1]$
   2. $i = i + 1$, $i_1 = i_1 + 1$
6. EndWhile
7. While $i_2 \leq U_2$ do //To copy the array A_2 into last part of A
   1. $A[i] = A_2[i_2]$
   2. $i = i + 1$, $i_2 = i_2 + 1$
8. EndWhile
9. Stop

Copyrighted material

Assignment 2.7 Modify the algorithm MERGE for the following cases of (i) if both A_1 and A_2 are empty; (ii) either A_1 or A_2 is empty.

2.3.3 Application of Arrays

There are wide applications of arrays in computation. This is why almost every programming language include this data type as a built-in data type. A simple example that can be cited is to store some data before performing their manipulation. Another simple example is mentioned as below:

Suppose, we want to store records of all students in a class. The record structure is given by

STUDENTS

ROLL NO	MARK1	MARK2	MARK3	TOTAL	GRADE
(Alphanumeric)	(Numeric)	(Numeric)	(Numeric)	(Numeric)	(Character)

If sequential storage of records is not an objection, then we can store the records by maintaining 6 arrays whose size is specified by the total number of students in the class, as in Figure 2.8.

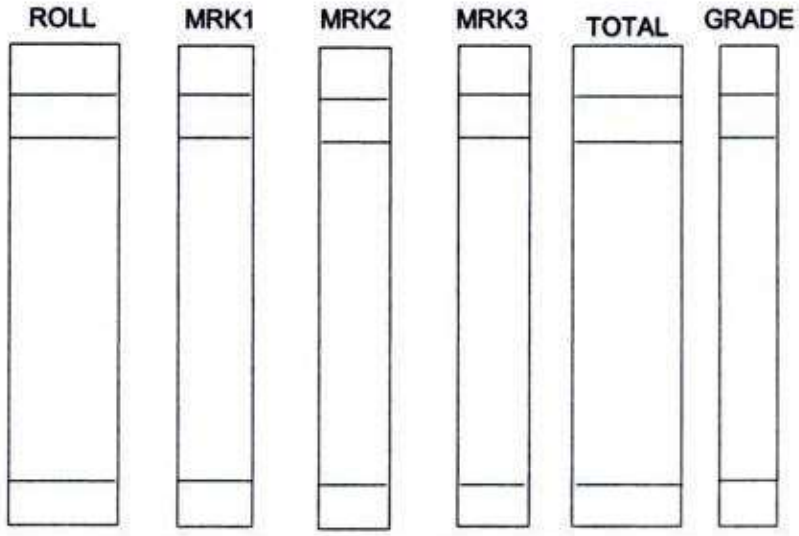


Fig. 2.8 Storing the students' records.

Assignment 2.8 (Practical)

1. Declare 6 arrays as required to store the students records each of size say, 30.
2. Initialize each array with appropriate data.
3. Use the following operations on your data structures
   (a) List all the records on the screen
   (b) Search for a record whose Roll No is say, X
   (c) Delete a record whose Roll No is say, X
   (d) Insert a new records just after the record, whose Roll No = X

Copyrighted material